Lesson 3: Pandas Basics

Cơ bản về Pandas

Lesson Information | Thông tin bài học

Item	Description
Duration	90 minutes
Type	Lecture + Lab
Prerequisites	Lesson 2: NumPy Advanced

Learning Objectives | Mục tiêu bài học

#	Objective	Assessment
1	Create and manipulate Series and DataFrames	Code exercises
2	Read data from various file formats	Lab work
3	Select and filter data effectively	Code exercises
4	Explore and understand datasets	Quiz

1. Introduction to Pandas (10 min)

1.1 What is Pandas?

Pandas is a powerful data manipulation library built on NumPy.

Key Features:

- DataFrame: 2D labeled data structure
- Series: 1D labeled array
- Data alignment and handling missing data
- GroupBy functionality
- Time series support

1.2 Installation and Import

```
# Install
pip install pandas

# Import
import pandas as pd
import numpy as np
```

```
print(pd.__version__)
```

---

2. Series (15 min)

2.1 Creating Series

```
# From list
s = pd.Series([1, 2, 3, 4, 5])
print(s)
# 0    1
# 1    2
# 2    3
# 3    4
# 4    5
# dtype: int64

# With custom index
s = pd.Series([1, 2, 3, 4], index=['a', 'b', 'c', 'd'])
print(s)
# a    1
# b    2
# c    3
# d    4

# From dictionary
data = {'a': 100, 'b': 200, 'c': 300}
s = pd.Series(data)
print(s)

# From NumPy array
s = pd.Series(np.random.rand(5))
```

2.2 Series Operations

```
s = pd.Series([10, 20, 30, 40, 50], index=['a', 'b', 'c', 'd', 'e'])

# Access by index
print(s['a'])        # 10
print(s[0])          # 10
print(s['a':'c'])    # a, b, c
print(s[0:3])        # First 3 elements

# Boolean indexing
print(s[s > 25])
# c    30
# d    40
# e    50
```

```
# Arithmetic
print(s + 10)        # Add 10 to all
print(s * 2)         # Multiply by 2

# Attributes
print(s.index)       # Index(['a', 'b', 'c', 'd', 'e'])
print(s.values)      # [10 20 30 40 50]
print(s.dtype)       # int64
print(s.name)        # None

# Methods
print(s.sum())       # 150
print(s.mean())      # 30.0
print(s.describe())  # Summary statistics
```

---

3. DataFrame (25 min)

3.1 Creating DataFrames

```
# From dictionary
data = {
    'Name': ['An', 'Binh', 'Cuong', 'Dung'],
    'Age': [20, 21, 22, 23],
    'City': ['Hanoi', 'HCMC', 'Danang', 'Hanoi'],
    'GPA': [3.5, 3.8, 3.2, 3.6]
}
df = pd.DataFrame(data)
print(df)
#     Name  Age    City  GPA
# 0     An   20   Hanoi  3.5
# 1   Binh   21    HCMC  3.8
# 2  Cuong   22  Danang  3.2
# 3   Dung   23   Hanoi  3.6

# From list of dictionaries
data = [
    {'Name': 'An', 'Age': 20},
    {'Name': 'Binh', 'Age': 21},
    {'Name': 'Cuong', 'Age': 22}
]
df = pd.DataFrame(data)

# From NumPy array
arr = np.random.rand(3, 4)
df = pd.DataFrame(arr, columns=['A', 'B', 'C', 'D'])

# With custom index
df = pd.DataFrame(data, index=['s1', 's2', 's3', 's4'])
```

3.2 DataFrame Attributes

```
df = pd.DataFrame({
    'Name': ['An', 'Binh', 'Cuong'],
    'Age': [20, 21, 22],
    'GPA': [3.5, 3.8, 3.2]
})

print(df.shape)       # (3, 3)
print(df.columns)     # Index(['Name', 'Age', 'GPA'])
print(df.index)       # RangeIndex(start=0, stop=3, step=1)
print(df.dtypes)
# Name     object
# Age       int64
# GPA     float64
print(df.values)      # NumPy array
print(df.T)           # Transpose
```

3.3 Viewing Data

```
# First/last rows
print(df.head())      # First 5 rows
print(df.head(2))     # First 2 rows
print(df.tail())      # Last 5 rows
print(df.tail(3))     # Last 3 rows

# Random sample
print(df.sample(2))   # 2 random rows

# Info and describe
print(df.info())      # Column info, dtypes, memory
print(df.describe())  # Statistics for numeric columns

# Value counts
print(df['City'].value_counts())
# Hanoi     2
# HCMC      1
# Danang    1

# Unique values
print(df['City'].unique())
print(df['City'].nunique())  # Number of unique
```

---

4. Reading Data (20 min)

4.1 CSV Files

```
# Read CSV
df = pd.read_csv('data.csv')

# Common parameters
df = pd.read_csv('data.csv',
    sep=',',              # Delimiter
    header=0,             # Row number for header
    index_col=0,          # Column to use as index
    usecols=['A', 'B'],   # Columns to read
    nrows=100,            # Number of rows
    skiprows=5,           # Skip first n rows
    na_values=['NA', ''], # Values to treat as NaN
    encoding='utf-8',     # Encoding
    parse_dates=['date']  # Parse as datetime
)

# Save to CSV
df.to_csv('output.csv', index=False)
```

4.2 Excel Files

```
# Read Excel
df = pd.read_excel('data.xlsx')

# Read specific sheet
df = pd.read_excel('data.xlsx', sheet_name='Sheet1')

# Read multiple sheets
sheets = pd.read_excel('data.xlsx', sheet_name=['Sheet1', 'Sheet2'])
df1 = sheets['Sheet1']
df2 = sheets['Sheet2']

# Read all sheets
all_sheets = pd.read_excel('data.xlsx', sheet_name=None)

# Save to Excel
df.to_excel('output.xlsx', index=False, sheet_name='Data')

# Multiple sheets
with pd.ExcelWriter('output.xlsx') as writer:
    df1.to_excel(writer, sheet_name='Sheet1')
    df2.to_excel(writer, sheet_name='Sheet2')
```

4.3 JSON Files

```
# Read JSON
df = pd.read_json('data.json')
```

```
# Read JSON with orientation
df = pd.read_json('data.json', orient='records')

# Save to JSON
df.to_json('output.json', orient='records', indent=2)
```

4.4 SQL Databases

```
import sqlite3

# Connect to database
conn = sqlite3.connect('database.db')

# Read SQL query
df = pd.read_sql('SELECT * FROM table_name', conn)
df = pd.read_sql_query('SELECT * FROM users WHERE age > 20', conn)

# Read entire table
df = pd.read_sql_table('table_name', conn)

# Save to SQL
df.to_sql('new_table', conn, if_exists='replace', index=False)

conn.close()
```

4.5 Other Formats

```
# HTML tables
tables = pd.read_html('https://example.com/table.html')
df = tables[0]  # First table

# Pickle (Python serialization)
df.to_pickle('data.pkl')
df = pd.read_pickle('data.pkl')

# Parquet (columnar format, efficient)
df.to_parquet('data.parquet')
df = pd.read_parquet('data.parquet')

# Clipboard
df = pd.read_clipboard()
df.to_clipboard()
```

---

5. Selecting Data (20 min)

5.1 Column Selection

```
df = pd.DataFrame({
    'Name': ['An', 'Binh', 'Cuong', 'Dung'],
    'Age': [20, 21, 22, 23],
    'City': ['Hanoi', 'HCMC', 'Danang', 'Hanoi'],
    'GPA': [3.5, 3.8, 3.2, 3.6]
})

# Single column (returns Series)
print(df['Name'])
print(df.Name)  # Dot notation (if no spaces)

# Multiple columns (returns DataFrame)
print(df[['Name', 'Age']])
```

5.2 Row Selection

```
# By integer position (iloc)
print(df.iloc[0])          # First row (Series)
print(df.iloc[0:3])        # Rows 0-2
print(df.iloc[[0, 2, 3]])  # Specific rows

# By label (loc)
df.index = ['s1', 's2', 's3', 's4']
print(df.loc['s1'])        # Row with label 's1'
print(df.loc['s1':'s3'])   # Rows s1 to s3 (inclusive!)
print(df.loc[['s1', 's3']])
```

5.3 Row and Column Selection

```
# iloc - integer-based
print(df.iloc[0, 1])       # Row 0, Column 1
print(df.iloc[0:2, 1:3])   # Rows 0-1, Columns 1-2
print(df.iloc[:, 1])       # All rows, Column 1

# loc - label-based
print(df.loc['s1', 'Age'])
print(df.loc['s1':'s2', ['Name', 'Age']])
print(df.loc[:, 'Age'])    # All rows, Age column
```

5.4 Boolean Filtering

```
df = pd.DataFrame({
    'Name': ['An', 'Binh', 'Cuong', 'Dung'],
    'Age': [20, 21, 22, 23],
    'City': ['Hanoi', 'HCMC', 'Danang', 'Hanoi'],
```

```
    'GPA': [3.5, 3.8, 3.2, 3.6]
})

# Single condition
print(df[df['Age'] > 21])

# Multiple conditions (use & for AND, | for OR)
print(df[(df['Age'] > 20) & (df['GPA'] > 3.5)])
print(df[(df['City'] == 'Hanoi') | (df['City'] == 'HCMC')])

# isin - check if in list
print(df[df['City'].isin(['Hanoi', 'HCMC'])])

# String methods
print(df[df['Name'].str.startswith('A')])
print(df[df['Name'].str.contains('ng')])

# Query method (SQL-like)
print(df.query('Age > 21 and GPA > 3.5'))
print(df.query('City == "Hanoi"'))
```

5.5 Selecting with Conditions

```
# where - keep values where condition is True
print(df['GPA'].where(df['GPA'] > 3.5))  # NaN where False

# mask - opposite of where
print(df['GPA'].mask(df['GPA'] > 3.5))   # NaN where True

# np.where for conditional assignment
df['Status'] = np.where(df['GPA'] >= 3.5, 'Good', 'Average')
```

Practice Exercises | Bài tập thực hành

Exercise 3.1: Create DataFrames

```
# Create a DataFrame with:
# - 10 students
# - Columns: Name, Math, English, Physics scores (random 0-100)
# - Calculate total and average
# - Find students with average > 80
```

Exercise 3.2: Read and Explore

```
# Download Titanic dataset from Kaggle or use:
# df =
pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/ma
ster/titanic.csv')

# Answer:
# 1. How many rows and columns?
# 2. What are the column names and types?
# 3. How many missing values per column?
# 4. What is the survival rate?
# 5. Average age of passengers?
```

Exercise 3.3: Selection Practice

```
# Using Titanic dataset:
# 1. Select Name and Age columns
# 2. First 10 passengers
# 3. Female passengers only
# 4. Passengers aged 20-30
# 5. First class passengers who survived
# 6. Passengers with 'Mrs' in name
```

Exercise 3.4: Create from Multiple Sources

```
# 1. Create DataFrame from dictionary
# 2. Save to CSV
# 3. Read back from CSV
# 4. Save to JSON
# 5. Compare original and loaded data
```

Summary | Tóm tắt

Key Concepts

Concept	Description
Series	1D labeled array
DataFrame	2D labeled data structure
Index	Row labels
Columns	Column labels

Selection Methods

Method	Usage
`df['col']`	Select column
`df[['a', 'b']]`	Select multiple columns
`df.iloc[0]`	Select by position
`df.loc['a']`	Select by label
`df[df['x'] > 0]`	Boolean filtering
`df.query()`	SQL-like filtering

Reading Functions

Function	Format
`read_csv()`	CSV files
`read_excel()`	Excel files
`read_json()`	JSON files
`read_sql()`	SQL databases

Next Lesson | Bài tiếp theo

Lesson 4: Pandas Data Manipulation

- Data cleaning
- Transformations
- GroupBy operations
- Merging and joining